

Full Stack Development with MERN

1. Introduction

- **Project Title:** Book A Doctor
- **Team Members:** Rajesh Kumar Kakara.

2. Project Overview

- **Purpose:** The purpose of this project is to provide a seamless platform bridging the gap between patients and healthcare professionals. It allows patients to easily browse verified doctors, book appointments, and manage their healthcare schedules. For administrators, it provides tools to manage users and doctor approvals.
- **Features:** User Registration and Login (Patient & Admin)
- Role-based access control (Admin, Patient, Doctor)
- Doctor application system for users wanting to register as practitioners
- Admin dashboard for user, doctor, and appointment management
- Doctor approval/rejection workflow
- Real-time appointment booking system with date and time selection
- Appointment management for doctors (approve/reject)
- Notification system for users regarding appointment and application status

3. Architecture

- **Frontend:** The frontend is built using React.js (bootstrapped with Vite) for a fast, component-based UI. It utilizes React Router for navigation and Context API for global state management (Authentication). The styling is custom CSS for a modern, responsive design.
- **Backend:** The backend is powered by Node.js and Express.js, providing a robust RESTful API architecture. It handles routing, middleware (authentication, error handling), and business logic.
- **Database:** MongoDB is used as the primary database (currently configured as an In-Memory MongoDB for development/testing). Mongoose ODM is used to define schemas and interact with the database.

4. Setup Instructions

Prerequisites:

Node.js
Npm
Git

Installation:

1. Clone the repository: `git clone https://github.com/RajeshKumar191205/BOOK-A-DOCTOR.git`
2. Navigate into the project directory: `cd BOOK-A-DOCTOR`
3. Install backend dependencies:

- cd server
- npm install
- 4. Install frontend dependencies:
 - cd ../client
 - npm install
- 5. Set up environment variables (if switching to a persistent database later).
Currently, the app uses an In-Memory MongoDB which requires zero configuration.

5. Folder Structure

Client: The React frontend is located in the client/ directory.

- assets/: Images and static assets.
- components/: Reusable React components (Navbar, Sidebar, Modals, Forms).
- context/: Context API files (e.g., AuthContext.jsx for auth state).
- pages/: Main view components (Home, Login, Register, AdminDashboard, UserDashboard).
- src/index.css & src/App.css: Global and component-specific styles.

Server: The Node.js backend is located in the server/ directory.

- config/: Database connection logic (db.js).
- controllers/: Business logic for handling API requests (authController, userC, adminC, doctorC).
- middlewares/: Custom middleware (e.g., authMiddleware.js for JWT verification).
- models/ & schemas/: Mongoose schemas defining the database structure.
- routes/: API endpoint definitions (authRoutes, userRoutes, adminRoutes, doctorRoutes).
- index.js: Entry point of the backend server.
Explain the organization of the Node.js backend.

6. Running the Application

Frontend: Open a terminal, navigate to the client directory, and run:

```
cd client
```

```
npm run dev (Starts Vite dev server on port 5173)
```

Backend: Open a second terminal, navigate to the server directory, and run:

```
cd server
```

```
node index.js (Starts Node server on port 8000)
```

7. API Documentation

Auth Routes (/api/auth)

- POST /register: Register a new user. Expects name, email, password. Returns user object and token.
- POST /login: Authenticate a user. Expects email, password. Returns user object and token.

Admin Routes (/api/admin)

- GET /getallusers: Fetch all registered users. (Requires Admin Auth)
- GET /getalldoctors: Fetch all doctor applications. (Requires Admin Auth)
- POST /changedoctorstatus: Approve/Reject a doctor. Expects doctorId, status. (Requires Admin Auth)

Doctor Routes (/api/doctor)

- GET /getalldoctors: Fetch all approved doctors (Public).
- POST /changeappointmentstatus: Approve/reject an appointment. (Requires Doctor Auth)

8. Authentication

Authentication is handled using JSON Web Tokens (JWT)

Upon successful login or registration, the backend generates a JWT

The frontend stores this token in localStorage and includes it in the Authorization header (Bearer <token>) for all subsequent protected API requests.

The backend authMiddleware intercepts these requests, verifies the token signature, and extracts the user ID to authorize access to specific routes (e.g., ensuring only admins can access /api/admin/* routes).

9. User Interface

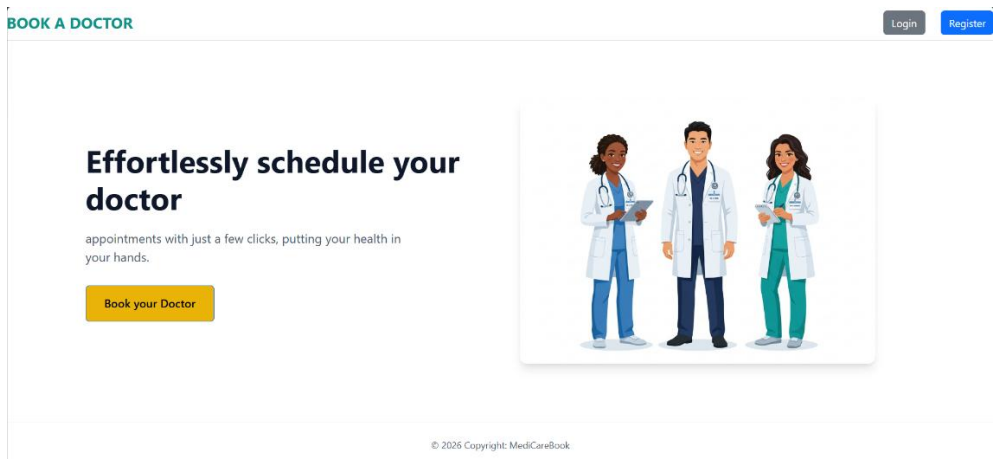
- The UI is built to be clean, modern, and intuitive.
- Features a responsive sidebar navigation for logged-in users.
- Utilizes a card-based layout for displaying doctor profiles.
- Modals are used for booking appointments to keep the user in context

10. Testing

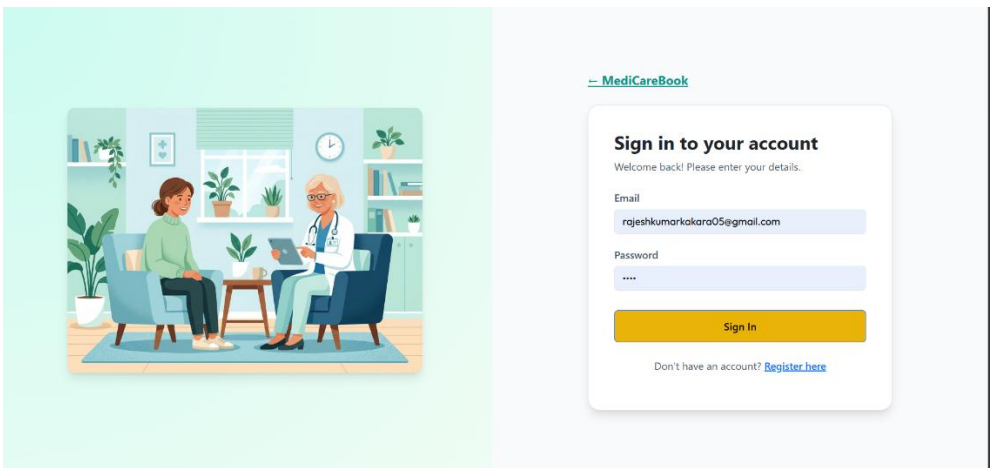
- **Strategy: Manual functional testing of core user flows (Registration, Login, Booking, Admin Approvals).**
- **Tools Used:**
- **Browser Developer Tools (Network tab for API inspection).**
- **Postman (for testing backend API endpoints in isolation).**

11.Screenshots or Demo

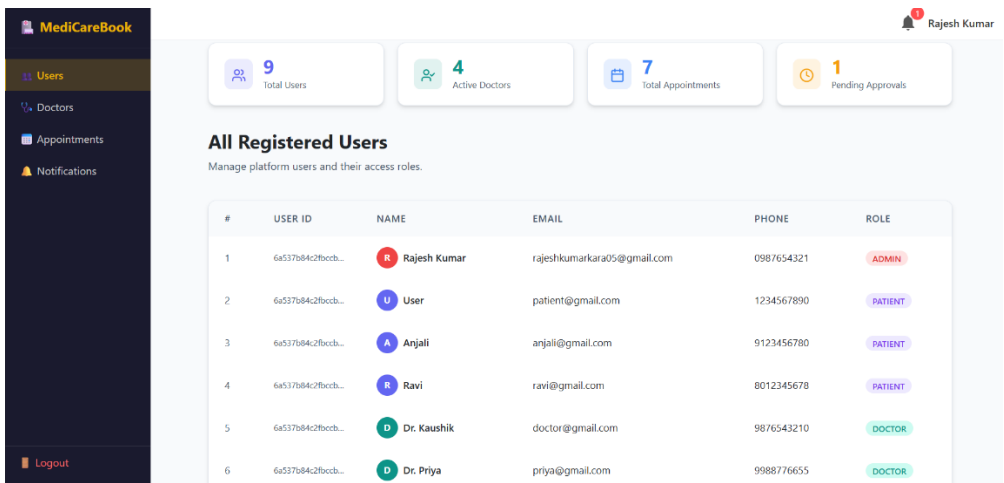
Home Page:



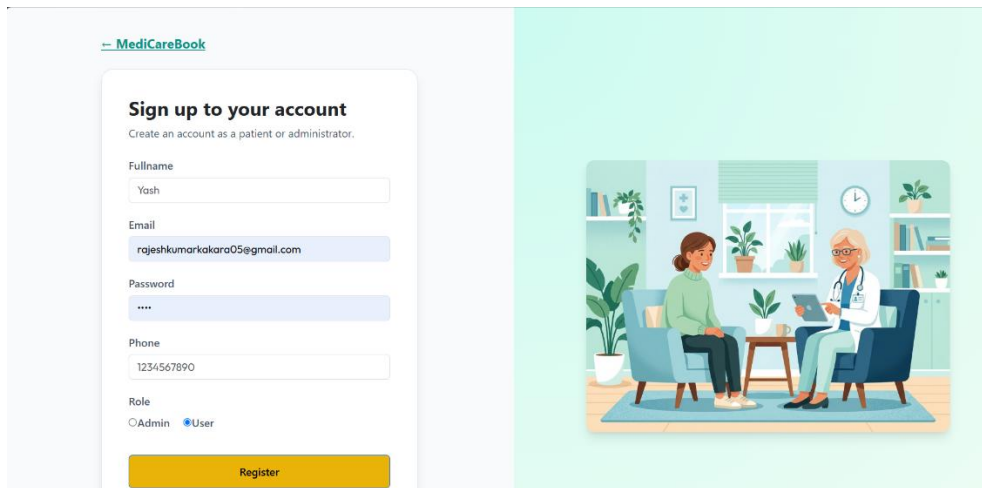
Login page:



User Interface:



Sign in:



The image shows the 'Sign up to your account' form on the MediCareBook website. The form is titled 'Sign up to your account' with a subtitle 'Create an account as a patient or administrator.' It includes input fields for Fullname (Yash), Email (rajeshkumarkakara05@gmail.com), Password (masked with ****), and Phone (1234567890). There are radio buttons for Role (Admin, User) with 'User' selected. A yellow 'Register' button is at the bottom. To the right is an illustration of a doctor and a patient sitting on a couch in a modern, bright room with plants and a clock.

MediCareBook

Sign up to your account
Create an account as a patient or administrator.

Fullname
Yash

Email
rajeshkumarkakara05@gmail.com

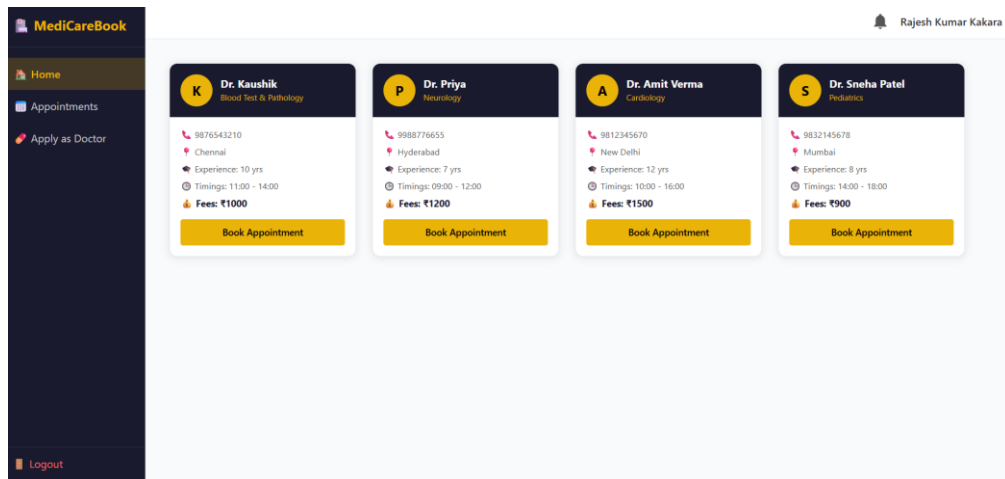
Password

Phone
1234567890

Role
☐ Admin ☒ User

Register

Doctor appointment:



The image shows the 'Doctor Appointment' page on the MediCareBook website. The page has a dark sidebar with 'Home', 'Appointments', 'Apply as Doctor', and 'Logout'. The main content area shows a grid of four doctor profiles, each with a circular icon, name, specialization, contact info, experience, timings, fees, and a 'Book Appointment' button. The user 'Rajesh Kumar Kakara' is logged in.

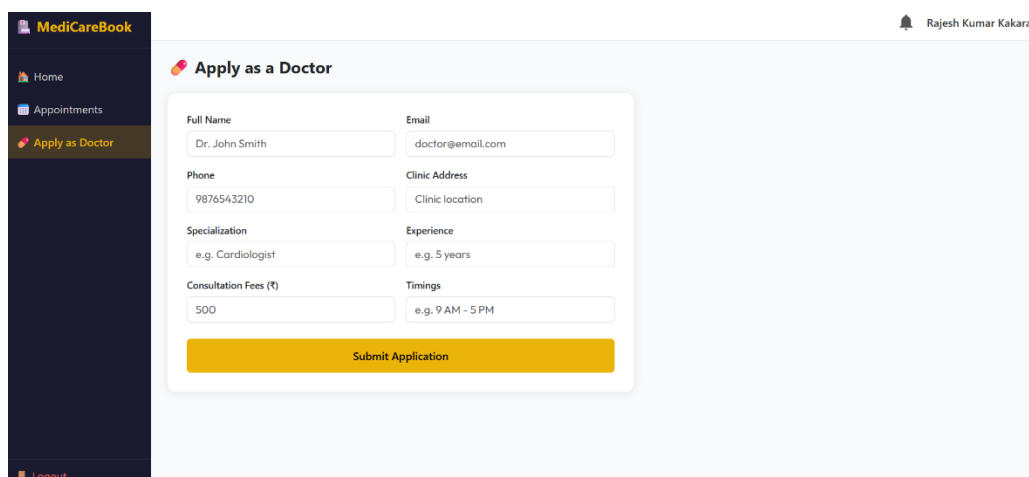
MediCareBook

Home
Appointments
Apply as Doctor
Logout

Rajesh Kumar Kakara

Dr. Kaushik Blood Test & Pathology	Dr. Priya Neurology	Dr. Amit Verma Cardiology	Dr. Sneha Patel Pediatrics
9876543210 Chennai Experience: 10 yrs Timings: 11:00 - 14:00 Fees: ₹1000	9888776655 Hyderabad Experience: 7 yrs Timings: 09:00 - 12:00 Fees: ₹1200	9812345670 New Delhi Experience: 12 yrs Timings: 10:00 - 16:00 Fees: ₹1500	9832145678 Mumbai Experience: 8 yrs Timings: 14:00 - 18:00 Fees: ₹900
Book Appointment	Book Appointment	Book Appointment	Book Appointment

Doctor Registration:



The image shows the 'Apply as a Doctor' form on the MediCareBook website. The form is titled 'Apply as a Doctor' and includes input fields for Full Name (Dr. John Smith), Email (doctor@email.com), Phone (9876543210), Clinic Address (Clinic location), Specialization (e.g. Cardiologist), Experience (e.g. 5 years), Consultation Fees (₹) (500), and Timings (e.g. 9 AM - 5 PM). A yellow 'Submit Application' button is at the bottom. The user 'Rajesh Kumar Kakara' is logged in.

MediCareBook

Home
Appointments
Apply as Doctor
Logout

Rajesh Kumar Kakara

Apply as a Doctor

Full Name
Dr. John Smith

Email
doctor@email.com

Phone
9876543210

Clinic Address
Clinic location

Specialization
e.g. Cardiologist

Experience
e.g. 5 years

Consultation Fees (₹)
500

Timings
e.g. 9 AM - 5 PM

Submit Application

12. Known Issues

The current database implementation uses mongodb-memory-server for ease of local testing. Data will be lost upon restarting the backend server. For production, this needs to be swapped to a persistent MongoDB instance

13.Future Enhancements

- Persistent Database Integration: Migrate from In-Memory MongoDB to MongoDB Atlas.
- Payment Gateway Integration: Integrate Stripe or Razorpay to handle consultation fees during the booking process.
- Video Consultation: Add WebRTC integration to allow virtual appointments directly through the platform.
- Email Verification: Implement real email verification using Nodemailer during registration.
- Password Reset: Add forgot/reset password flows.
- Advanced Search: Add filters for doctors based on specialty, location, and rating.